MS2 Summary – Zurich Footfall Forecast

What I achieved in this milestone

For MS2 I implemented the complete feature pipeline for the Zurich Footfall Forecast project.

The pipeline now has two main modes:

- The historical backfill mode builds training features for a selected historical time period.
- The live mode downloads the newest available Hystreet data and the current Open-Meteo weather forecast and creates the latest feature row for future predictions

The main feature pipeline is implemented in src/footfall/features. The pipeline now created the following feature:

- pedestrian count 24 hours ago
- pedestrian count 168 hours ago
- rolling pedestrian average over 3, 6, 24 hours
- hour of day
- day of week
- weekend indicator
- month
- zurich public holiday indicator
- street Parade impact indicator
- sechseläuten indicator
- weather forecast temperature for the target hour
- weather forecast precipitation for the target hour
- weather forecast weather code for the target hour

One important change compared with MS1 is the weather logic.

The proposal originally planned to use current weather information. This was changed because the model predicts 24 hours into the future. Historical training features now use archived 24-hours-ahead weather forecast from the Open-Meteo Previous Runs API.

The live pipeline uses the current Open-Meteo Forecast API and selects the weather forecast for the target timestamp, which is 24hours after the prediction timestamp.

The feature store is implemented using versioned Parquet files.

- Training snapshots are stored under data/features/training
- Live snapshots are stored under data/features/live

A fully historical backfill from 20.01.2024 to 31.10.2026 produced 24’384 hourly feature rows.

During the data-quality check I found 52 rows with a missing forecast temperature and one target row that was still marked unverified. These rows remain in the feature store They will be filtered before model training in MS3.

The live feature pipeline is automated with GitHub Actions. The workflow is configured to run approximately every three hours. It installs the projects dependencies, runs the tests, runs the live feature pipeline and uploads the generated Parquet files as a GitHub actions artifact.

The project is also containerized with Docker. I successfully tested both the live pipeline and the historical backfill inside the Docker container.

Dependencies are reproducible through pyproject.toml and uv.lock. Unit tests were added for the most important feature logic and feature storage. The most important files for reviewers are:

- README.md
- src/footfall/features/pipeline.py
- src/footfall/features/build_features.py
- src/footfall/features/ingest_weather.py
- src/footfall/features/store.py
- .github/workflows/feature_pipeline.yaml
- tests/
- Dockerfile

Response to MS1 reviews

A1 – The MAE target of 200 was not supported by baseline results

- Status: Not yet
- The seasonal-naive baseline MAE, mean hourly pedestrian count and peak pedestrian count have not yet been calculated
- This will be done in MS3 because model evaluation and baseline comparison belong to the training pipeline.
- The current success criterion will be reviewed after the baseline results are available

A2 / B1 – Current weather was used instead of the weather forecast for the target hour

- Status Changed:
- The feature pipeline now uses the weather forecast for the target hour
- Historical training features use archived 24-hour-ahead forecast from the Open-Meteo Previous Runs API
- The live pipeline uses the current Open-Meteo Forecast API
- Both pipelines create the same weather features:
    - forecast_temperature
    - forecast_precipitation
    - forecast_weather_code
- Relevant files:
    - src/footfall/features/ingest_weather.py
    - src/footfall/features/build_features.py
    - config/settings.yaml

A3 / B2 – Holidays and important evets were missing

- Status: Changed
- A holiday feature for the target day was added using the Python holiday package for Switzerland and the canton of Zurich
- Event features were also added for Street Parade impact data and Sechseläuten
- Relevant Files:
    - src/footfall/features/build_features.py
    - config/settings.yaml

A4 – The 1-hour and 2-hour lag features may not be available or verified during live inference

- Status: Changed
- The 1-hour and 2-hour lag features were removed
- The pipeline now uses 24-hour and 168-hour pedestrian lag features
- The rolling features also end 24 hours before the prediction timestamp.
- Unverified pedestrian observations are excluded from the rolling calculations
- This makes the feature set more consistent wit the three-hour polling schedule and reduces train-serving skew.
- Relevant files:
    - src/footfall/features/build_features.py
    - config/settings.yaml

B3 – Previous-year pedestrian features could capture long-term trends

- Status: Not yet
- A previous year feature wat not added in MS2
- I plan to test this feature during MS3 and only keep it if it improves model performance

B4 – Persist MLflow tracking data and explain stack choices

- Status: Partly
- Persistent MLflow tracking is not part of MS2 and will be decided during MS3, before the training pipeline is automated
- The current MS2 feature store intentionally uses versioned Parquet because it's simple, inspectable, reproducible and explicitly sufficient for the feature-pipeline milestone.
- Scheduled GitHub Actions artifacts are currently temporary storage for live runs. A persistent cloud store such as GCS is planned for later deployment
- The MLFlow backend / registry persistence locations remains MS3 decision

E1 – Same sensor/sources as another HS26 project

- Status: Partly
- The project keeps the hourly 24-hour ahead Bahnhofstrasse Mitte target but now differentials itself through P10 / P50 / P90 quantile forecasts for staffing decisions, rather than only producing a single point forecast
- This differentiator is documented in README.md
- The required feature pipeline is already compatible with this design, but the quantile models themselves belong to MS3 and are therefore not yet implemented.
- The editorial summary explicitly suggested prediction intervals as an acceptable differentiation strategy

E2 – Specify rolling windows and use timestamp-based rather than positional lags

- Status: Changed
- The implemented rolling windows are explicitly 3h, 6h and 24h. Each ending at t-24h.
- Pedestrian lag features are created using timestamp-based joins rather than positional row offsets. The Hystreet ingestion additionally validates hourly continuity.
- Relevant files:
    - src/footfall/features/build_features.py
    - src/footfall/features/ingest.py
    - config/settings.yaml